

StoreSignal

AI Representation Optimizer for Shopify Merchants

Product Document

Understand how AI shopping agents see your store. Fix gaps. Get recommended.

Team Aryan Rout Sambhav Sahoo	Track Track 5 — Advanced AI Representation Optimizer	Hackathon Kasparro Agentic Commerce May 2026
--	---	---

1. The Problem — and Why It Matters to Us

We kept coming back to a simple question while reading the hackathon brief: if a merchant spends months building a Shopify store, writing their descriptions, setting their policies — why would an AI agent ignore them?

The answer, once we dug in, is surprisingly mundane. AI shopping agents don't experience a store the way a human does. They don't see the beautiful product photos or the thoughtfully designed homepage. They read raw text — product descriptions, policy pages, FAQ content, review text, and structured metadata. They form an internal summary of what the store is and what it offers. Then they decide whether to recommend it.

If that raw text is vague, incomplete, or contradictory — the AI either skips the merchant or describes them poorly. And the merchant never finds out why. They just notice that their AI-channel sales are zero, or worse, that customers are being told the wrong thing about their return policy.

The problem is not that merchants have bad products. The problem is that their store data is not readable by machines — and nobody has built a tool that shows them this in plain language.

When an AI agent encounters a poorly structured store, three things can happen — none of them good:

- The merchant gets skipped. The AI lacks enough confident information to recommend them, so it recommends a competitor whose data is clearer.
- The merchant gets misrepresented. The AI guesses at details it cannot find — stating wrong shipping times, incorrect return conditions, or fabricated product specs.
- The merchant gets undersold. They get a passing mention — 'there is a store that sells bags' — with none of the specificity that converts a browser into a buyer.

All three outcomes mean lost revenue. None of them are visible to the merchant. That invisibility is what we set out to fix.

2. Who We Built This For

We spent time thinking about who actually has this problem before writing a single line of code. The answer is not a big brand with a dedicated e-commerce team. Big brands have people who handle SEO, structured data, and content optimization. They will adapt to AI channels when the time comes.

The person who really has this problem is a small independent merchant — someone who chose Shopify specifically because they wanted to own their store, not hand it to Amazon. They

know their products deeply. They care about their customers. They built something real. But they wrote their product descriptions the same way they would write a WhatsApp message to a friend — warm and enthusiastic, but thin on the kind of specific detail that an AI agent can use.

About the user	What we observed
Who they are	Independent Shopify store owner. Clothing, leather goods, electronics, food, handmade products. Usually solo or small team.
Technical comfort	Comfortable using Shopify's dashboard. Not comfortable with JSON-LD, schema markup, or structured data. The word 'metadata' means nothing to them.
Their current frustration	They get traffic from Google but see zero engagement from AI-driven channels. They have no idea why. They suspect their descriptions might be the problem but do not know what to change.
What they actually need	Not a technical report. Not a list of 40 problems. A clear score, a plain-language explanation of how AI currently sees their store, and fixes they can copy and paste in the next 20 minutes.
What they are afraid of	Spending hours rewriting content based on advice that turns out to be generic and useless. They have been burned by SEO consultants before.

Before StoreSignal, this merchant's options were: read a generic blog post, hire a consultant, or guess. None of those options give them a store-specific diagnosis. None of them show them what an AI actually says about their store. That is the gap we are filling.

3. What We Built — and the Journey We Designed

We went back and forth on scope a lot during the first day. We could have built a simple checklist tool — 'do you have a return policy? yes/no' — and called it an audit. But that felt dishonest. A store can have a return policy that says 'we accept returns' and still be completely useless to an AI agent trying to tell a customer whether their purchase is protected.

The insight that shaped everything: the only way to really know how an AI sees a store is to ask an AI. So that became the core of the product — not a checklist, but a simulation.

StoreSignal is built around three actions: Diagnose — Fix — Preview. Everything else is in service of that loop.

The user journey we designed

Step	Merchant experience	What happens in the system
------	---------------------	----------------------------

1	Connects their Shopify store	OAuth flow, Admin API fetches products, descriptions, policies, FAQs, reviews, metadata
2	Sees their AI Readiness Score	Hybrid engine runs — deterministic audit + OpenAI semantic analysis — produces score across 6 dimensions
3	Reads how AI currently sees their store	LLM generates a realistic simulation: exactly what an AI shopping agent would say about this store today
4	Sees the gap	Side-by-side: how the merchant wants to be described vs. how AI currently describes them
5	Reviews what is broken and why	Ranked issue list with severity tags, explanation of why each issue hurts AI recommendation chances
6	Gets fixed versions	AI rewrites their product descriptions, return policy, and generates a store-specific FAQ — all copy-paste ready
7	Sees the improvement	Before vs. after comparison showing new AI perception score and updated perception simulation

The journey is intentionally linear. We did not want merchants to land on a dashboard with 15 things happening at once. They connect, they see the problem clearly, they get the fix, they see the improvement. That is it.

4. The Decisions That Shaped the Product

Using AI to simulate AI perception — not just rules

This was the first and most important decision. A rule-based tool can check whether a return policy exists. It cannot tell you whether that return policy is clear enough for an AI agent to confidently relay to a buyer. That requires actual semantic understanding — reading the text the way a language model reads it and judging whether the meaning is unambiguous.

We briefly considered doing everything with rules because it would be faster and more predictable. But we realised we would be building a different product — a completeness checker, not an AI representation optimizer. The whole point is to show merchants how AI sees them. The only honest way to do that is to actually ask an AI.

Making the perception gap visible — not just the score

Early on we had a version of the product that just showed a score — 47 out of 100 — with a list of issues below it. We showed it to a few people and they all asked the same question: 'But what does the AI actually say about me?'

That question told us the score alone was not enough. The most valuable thing we could show a merchant is not a number but a sentence — the exact words an AI agent would use to

describe their store. When a merchant reads 'This appears to be a bag retailer with limited product information and no clear return process,' they understand the problem immediately. No explanation needed. We made this the centrepiece of the dashboard.

Ranking fixes by impact, not just severity

We had a long internal debate about how to rank the action plan. Severity-based ranking — critical, medium, low — felt logical. But two issues can both be critical and have very different impact on whether an AI actually recommends the store.

We settled on impact-based ranking because merchants have limited time. The first thing they see when they open the action plan should be the one fix that will move the needle the most. We derived impact scores from our understanding of which data signals AI shopping agents weight most heavily when forming recommendations. Return policy clarity and product description specificity consistently came out at the top.

Explaining every score, not just reporting it

We had a version without explanations — cleaner UI, faster rendering. We cut it because without explanations, the tool asks merchants to trust a number they do not understand. A score of 34 means nothing without knowing that 22 of those lost points came from vague product descriptions and 18 came from an unclear return policy.

Explainability is not a feature we added on top. It is part of the core value. A merchant who understands why they scored 34 knows exactly what to do. A merchant who just sees 34 either ignores it or panics. We chose the former outcome.

Generating the fix, not just identifying the problem

This was a scope decision that almost did not make the cut. Rewriting descriptions and policies with AI adds complexity — prompt engineering, quality control, handling edge cases. We seriously considered just flagging the problems and leaving rewriting to the merchant.

We decided against it because we watched what happens when tools only diagnose. Merchants understand the problem, they intend to fix it, and then life gets in the way. The gap between knowing and doing is where most improvements die. If we generate the improved version — and make it copy-paste ready — we remove that gap entirely. That is the difference between a diagnostic tool and a tool that actually changes things.

5. What We Chose Not to Build

We made a lot of cuts. Some were forced by time. Others were deliberate product decisions. Here is an honest account of both.

What we cut	Why
User accounts and authentication	Every analysis session is independent for now. Adding auth before the core loop was polished felt backwards. Build the thing worth logging into first.
Multi-store comparison	The value of comparison only exists after the single-store analysis is solid. Showing a merchant how they compare to competitors before they have fixed their own store is premature.
Real-time monitoring and alerts	Monitoring is a retention feature, not an acquisition feature. A merchant needs to trust the diagnosis before they care about ongoing monitoring. We focused on the first experience.
Browser extension	Interesting idea — analyze any store you are browsing. But it splits the use case. Merchants need a dedicated session to act on recommendations, not a sidebar popup while they are doing something else.
PDF report export	The dashboard serves the same purpose for the hackathon. This is a v2 feature that makes sense once merchants want to share reports with their team or agency.
Competitor benchmarking	Genuinely valuable but requires analyzing other merchants' stores, which raises privacy and scope questions we did not want to solve in 10 days.

The question we kept asking before adding anything: does this make the core loop better, or does it make the product bigger? We consistently chose better over bigger.

6. Tradeoffs We Had to Make

Consistency vs intelligence in scoring

Pure AI scoring is semantically rich but introduces variance — the same store could score differently across two runs depending on how the model interprets the prompt. Pure deterministic scoring is consistent but misses semantic quality entirely. A return policy that says 'we accept returns' would pass a rule-based check but fail a semantic one.

We landed on a hybrid: deterministic rules set the baseline for things that are binary (field exists or not, policy has timeframe or not), and the AI layer evaluates quality within those dimensions. This means the score is consistent — the same store will score the same way every time — while still capturing the semantic quality that makes the difference between a useful analysis and a checklist.

Speed vs quality in AI analysis

A single large prompt asking the AI to do everything at once is faster and cheaper. Chained prompts — separate calls for ambiguity detection, perception simulation, rewriting, and FAQ generation — are slower and more expensive but produce significantly better outputs.

We tested both approaches. The single-prompt version produced generic, template-like outputs — the kind of rewritten description that sounds polished but contains no store-specific information. The chained version produced outputs that felt genuinely tailored to the specific store being analyzed. For a product where the fix is the core value, we chose quality over speed without much hesitation.

Shopify API vs manual input

Manual input is much simpler to build — no OAuth, no API integration, no dependency on Shopify's infrastructure. We could have had a working demo on day one.

We chose the Shopify Admin API because manual input is a prototype, not a product. A tool that requires merchants to copy-paste their entire store's content before getting an analysis will not be used. The API integration is what makes this feel like a real product rather than a hackathon demo. It was harder to build and we hit rate limit issues we had to handle carefully, but we do not regret the decision.

7. What We Would Do Differently

If we had more time, the first thing we would improve is the ambiguity detection. Right now it relies on a combination of pattern matching and LLM analysis, which catches the obvious cases but misses subtler forms of vagueness. A description can be specific-sounding without being actually useful to an AI — lots of words that add up to nothing. Improving that detection layer is the single highest-leverage improvement we could make.

We would also add real-time monitoring — alerts when a merchant's AI readiness score drops because they updated a product description and accidentally made it worse. That is a genuinely useful feature that requires the core product to exist first, which is why we deferred it.

The thing we are most proud of is the perception simulation. Showing merchants exactly what an AI would say about their store — in plain language — turned out to be the moment in the demo where people really understood the problem. Everything else in the product exists to support that moment.

— End of Product Document —